

Practical Lab: DuckDB for Big Data Warehouse

สร้าง Dataset จำลอง • Analytical SQL • Data Warehouse Thinking • ทำงานบน Browser

LAB MODE: ใช้ DuckDB Web Shell ได้ที่ <https://shell.duckdb.org/> โดยไม่ต้องติดตั้งโปรแกรม

1. วัตถุประสงค์ของแลป

- เข้าใจความต่างระหว่างงาน Transaction (OLTP) กับงานวิเคราะห์ (OLAP)
- ทดลองสร้างข้อมูลจำนวนมากด้วย SQL โดยไม่ต้องดาวน์โหลด Dataset
- ฝึกใช้ SQL สำหรับงาน Data Warehouse เช่น GROUP BY, aggregation, CTE และ Window Function
- เห็นแนวคิด Fact Table, Dimension และ KPI จากสถานการณ์ธุรกิจจริง
- ทดลองแนวคิด Columnar Analytics ผ่าน DuckDB ซึ่งเหมาะกับ analytical workload

2. Scenario: PungFoo Mart Analytics

สมมติว่าเราเป็น Data Analyst ของร้านค้าออนไลน์ “PungFoo Mart” ซึ่งมีรายการขายจำนวนมากตลอดปี 2025 ผู้บริหารต้องการ Dashboard เพื่อดูยอดขาย สินค้าขายดี ลูกค้านี่มีมูลค่าสูง และแนวโน้มรายเดือน

Table	บทบาท	ตัวอย่างข้อมูล
products	Dimension	product_id, product_name, category, unit_price
customers	Dimension	customer_id, region, segment
sales	Fact	sale_id, order_date, product_id, customer_id, quantity, amount

3. เริ่มต้นใช้งาน DuckDB Online

1) เปิด Browser → เข้า <https://shell.duckdb.org/> 2) พิมพ์คำสั่ง SQL ในช่องคำสั่ง 3) กด Enter เพื่อรัน

```
SELECT 'Hello Data Warehouse!' AS message;
```

NOTE: หากต้องการเริ่มใหม่ สามารถ Refresh หน้า Web Shell ได้ ข้อมูลที่สร้างใน session เดิมอาจหายได้ ดังนั้นแลปนี้ออกแบบให้สร้าง Dataset ใหม่ได้รวดเร็ว

4. Part A — Generate Dataset ด้วย SQL

4.1 สร้าง Dimension: Products

```
CREATE OR REPLACE TABLE products AS
SELECT
  i AS product_id,
  'Product_' || LPAD(CAST(i AS VARCHAR), 3, '0') AS product_name,
  CASE
    WHEN i % 5 = 0 THEN 'Electronics'
    WHEN i % 5 = 1 THEN 'Food'
    WHEN i % 5 = 2 THEN 'Fashion'
    WHEN i % 5 = 3 THEN 'Home'
    ELSE 'Sport'
  END AS category,
  ROUND(50 + random() * 4950, 2) AS unit_price
FROM range(1, 101) t(i);

SELECT * FROM products LIMIT 10;
```

4.2 สร้าง Dimension: Customers

```
CREATE OR REPLACE TABLE customers AS
SELECT
  i AS customer_id,
  'Customer_' || LPAD(CAST(i AS VARCHAR), 4, '0') AS customer_name,
  CASE
    WHEN i % 4 = 0 THEN 'Bangkok'
    WHEN i % 4 = 1 THEN 'East'
    WHEN i % 4 = 2 THEN 'North'
    ELSE 'South'
  END AS region,
  CASE
    WHEN i % 3 = 0 THEN 'Premium'
    WHEN i % 3 = 1 THEN 'Regular'
    ELSE 'New'
  END AS status;
```

```
END AS segment
FROM range(1, 5001) t(i);

SELECT * FROM customers LIMIT 10;
```

4.3 สร้าง Fact Table: Sales จำนวน 100,000 รายการ

```
CREATE OR REPLACE TABLE sales AS
SELECT
  i AS sale_id,
  DATE '2025-01-01' + CAST(FLOOR(random() * 365) AS INTEGER) AS order_date,
  1 + CAST(FLOOR(random() * 100) AS INTEGER) AS product_id,
  1 + CAST(FLOOR(random() * 5000) AS INTEGER) AS customer_id,
  1 + CAST(FLOOR(random() * 5) AS INTEGER) AS quantity,
  ROUND(100 + random() * 9900, 2) AS amount
FROM range(1, 100001) t(i);

SELECT COUNT(*) AS total_rows FROM sales;

SELECT * FROM sales LIMIT 10;
```

NOTE: ทุกครั้งที่สร้าง Dataset ใหม่ ค่า random() จะทำให้ตัวเลขแตกต่างกันเล็กน้อย ดังนั้นคำตอบเชิงตัวเลขของนิสิตแต่ละคนอาจไม่เหมือนกัน ซึ่งเป็นเรื่องปกติ

5. Part B — Analytical SQL พื้นฐาน

5.1 KPI ภาพรวม

```
SELECT
  COUNT(*) AS transactions,
  SUM(quantity) AS units_sold,
  ROUND(SUM(amount), 2) AS revenue,
  ROUND(AVG(amount), 2) AS avg_order_value
FROM sales;
```

5.2 ยอดขายรายเดือน

```
SELECT
  DATE_TRUNC('month', order_date) AS month,
  COUNT(*) AS transactions,
  ROUND(SUM(amount), 2) AS revenue
FROM sales
GROUP BY 1
ORDER BY 1;
```

5.3 Join Fact + Dimension เพื่อหาหมวดสินค้าขายดี

```
SELECT
  p.category,
  COUNT(*) AS transactions,
  SUM(s.quantity) AS units_sold,
  ROUND(SUM(s.amount), 2) AS revenue
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

6. Part C — จาก SQL ธรรมดาสู่ Data Warehouse Analytics

6.1 วิเคราะห์ยอดขายตาม Region และ Customer Segment

```
SELECT
  c.region,
  c.segment,
  COUNT(*) AS transactions,
  ROUND(SUM(s.amount), 2) AS revenue
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
GROUP BY c.region, c.segment
ORDER BY c.region, revenue DESC;
```

6.2 Top 10 Products

```
SELECT
  p.product_name,
  p.category,
  ROUND(SUM(s.amount), 2) AS revenue
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.product_name, p.category
ORDER BY revenue DESC
LIMIT 10;
```

6.3 Window Function: จัดอันดับสินค้าภายในแต่ละ Category

```
WITH product_sales AS (
  SELECT
    p.category,
    p.product_name,
    SUM(s.amount) AS revenue
  FROM sales s
  JOIN products p ON s.product_id = p.product_id
  GROUP BY p.category, p.product_name
)
SELECT
  category,
  product_name,
  ROUND(revenue, 2) AS revenue,
  RANK() OVER (
    PARTITION BY category
    ORDER BY revenue DESC
  ) AS category_rank
FROM product_sales
QUALIFY category_rank <= 3
ORDER BY category, category_rank;
```

7. Challenge Lab — ให้นิสิตทำเอง

Q1 เดือนไหนมียอดขาย (Revenue) สูงที่สุด?

ตอบ เดือน 5

```
memory D SELECT
    DATE_TRUNC('month', order_date) AS month,
    ROUND(SUM(amount), 2) AS revenue
FROM sales
GROUP BY 1
ORDER BY revenue DESC
LIMIT 1;
```

month timestamp	revenue double
2025-05-01 00:00:00	43272002.29

Q2 Region ไตสร้างรายได้สูงที่สุด?

ตอบ East

```
memory D SELECT
    c.region,
    ROUND(SUM(s.amount), 2) AS revenue
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
GROUP BY c.region
ORDER BY revenue DESC
LIMIT 1;
```

region varchar	revenue double
East	127480596.35

Q3 Customer Segment ไตมี Average Order Value สูงที่สุด?

ตอบ Regular

```
LIMIT 1;
```

segment varchar	avg_order_value double
Regular	5053.6

Q4 หาลูกค้า Top 10 คนที่สร้างรายได้รวมสูงที่สุด พร้อม Region และ Segment

ตอบ

customer_id int64	customer_name varchar	region varchar	segment varchar	total_revenue double
1364	Customer_1364	Bangkok	New	206832.51
4841	Customer_4841	East	New	204981.31
4831	Customer_4831	South	Regular	204913.67
2369	Customer_2369	East	New	202788.3
3618	Customer_3618	North	Premium	201359.82
1532	Customer_1532	Bangkok	New	195050.62
2390	Customer_2390	North	New	192795.43
4009	Customer_4009	East	Regular	192208.32
3865	Customer_3865	East	Regular	191844.98
6	Customer_0006	North	Premium	190113.43

10 rows 5 columns

Q5 สำหรับแต่ละ Region ให้หาหมวดสินค้า (Category) ที่ทำรายได้สูงสุดอันดับ 1
ตอบ

region varchar	category varchar	revenue double
Bangkok	Sport	25844291.48
East	Food	25723253.65
North	Home	25601702.4
South	Food	25192632.12

Q6 Bonus: เปลี่ยน Fact Table จาก 100,000 rows เป็น 1,000,000 rows
แล้วเปรียบเทียบความรู้สึกด้านความเร็วในการ Query
ตอบ

```
memory D CREATE OR REPLACE TABLE sales AS
SELECT
  i AS sale_id,
  DATE '2025-01-01' + CAST(FLOOR(random() * 365) AS INTEGER) AS order_date,
  1 + CAST(FLOOR(random() * 100) AS INTEGER) AS product_id,
  1 + CAST(FLOOR(random() * 5000) AS INTEGER) AS customer_id,
  1 + CAST(FLOOR(random() * 5) AS INTEGER) AS quantity,
  ROUND(100 + random() * 9900, 2) AS amount
FROM range(1, 1000001) t(i);
memory D SELECT COUNT(*) AS total_rows FROM sales;
```

total_rows int64
1000000

8. Hint สำหรับ Challenge

-- ตัวอย่างโครงสร้าง Query ที่นำไปประยุกต์ได้

```
SELECT
  <dimension>,
  COUNT(*) AS transactions,
  SUM(amount) AS revenue,
  AVG(amount) AS avg_order_value
FROM sales
JOIN <dimension_table> ON ...
GROUP BY <dimension>
ORDER BY revenue DESC;
```

9. ทดลอง Scale Up: จาก 100K → 1M Rows

ให้เปลี่ยน range(1, 100001) ในคำสั่งสร้าง sales เป็น range(1, 1000001) แล้วรัน Query KPI และ GROUP BY ซ้ำ เพื่อสังเกตพฤติกรรมของ analytical database

```
SELECT COUNT(*) FROM sales;
```

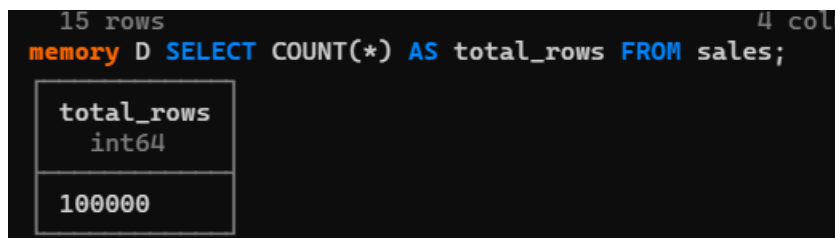
- DuckDB ถูกออกแบบมาสำหรับ analytical workload และประมวลผลแบบ column-oriented/vectorized
- งานลักษณะนี้แตกต่างจาก OLTP ที่เน้น INSERT/UPDATE ราย transaction
- แนวคิดเดียวกันสามารถต่อยอดไปยัง Cloud Data Warehouse เช่น BigQuery, Snowflake และ Redshift ได้

10. Discussion: เชื่อมโยงกับ Cloud Data Warehouse

ประเด็น	DuckDB	Cloud DW	แนวคิดที่เหมือนกัน
การติดตั้ง	รันในเครื่อง/Browser	บริการบน Cloud	ใช้ SQL วิเคราะห์ข้อมูล
Scale	เหมาะกับ local / single node	Scale ได้ระดับองค์กร	เน้น analytical workload
ค่าใช้จ่าย	ฟรี / Open source	คิดตาม Compute/Storage/Usage	ต้องคิดเรื่อง query efficiency
ตัวอย่าง	DuckDB	BigQuery / Snowflake / Redshift	Aggregation, Join, OLAP

11. สิ่งที่ทำให้มันดี

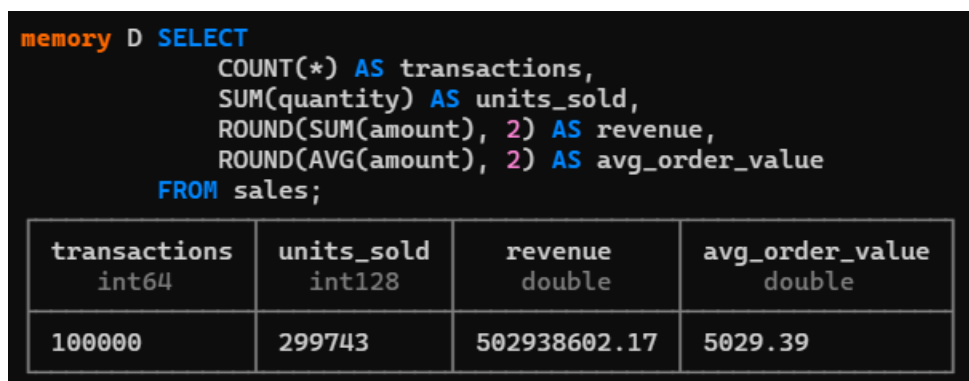
- Screenshot ผลลัพธ์ Dataset ที่มีอย่างน้อย 100,000 rows



```
15 rows 4 columns
memory D SELECT COUNT(*) AS total_rows FROM sales;
```

total_rows
int64
100000

- Screenshot Query KPI ภาพรวม



```
memory D SELECT
COUNT(*) AS transactions,
SUM(quantity) AS units_sold,
ROUND(SUM(amount), 2) AS revenue,
ROUND(AVG(amount), 2) AS avg_order_value
FROM sales;
```

transactions	units_sold	revenue	avg_order_value
int64	int128	double	double
100000	299743	502938602.17	5029.39

- SQL + Result ของ Challenge Q1–Q5
- สรุป 3–5 บรรทัดว่า DuckDB เหมาะกับงาน Data Warehouse/Analytics อย่างไร

ตอบ เพราะ DuckDB มีสถาปัตยกรรมแบบ Columnar Storage และ Vectorized Execution ที่อ่านและประมวลผลเฉพาะคอลัมน์ที่จำเป็น ทำให้คำนวณ Aggregation และ Join ข้อมูลปริมาณมากได้อย่างรวดเร็ว นอกจากนี้ยังเป็น In-Process Database ที่ใช้งานได้ทันทีโดยไม่ต้องติดตั้ง Server รองรับ SQL วิเคราะห์ข้อมูลขั้นสูงเต็มรูปแบบ เหมาะสำหรับการทำ Local Analytics, Data Exploration และต่อยอดไปยัง Cloud Data Warehouse เช่น BigQuery หรือ Snowflake ได้อย่างมีประสิทธิภาพ

12. Instructor Notes / เฉลยเชิงแนวคิด

- Q1: GROUP BY DATE_TRUNC('month', order_date) แล้ว ORDER BY SUM(amount) DESC LIMIT 1
- Q2: JOIN customers แล้ว GROUP BY region
- Q3: JOIN customers แล้ว GROUP BY segment และใช้ AVG(amount)
- Q4: JOIN customers, GROUP BY customer_id/customer_name/region/segment แล้ว ORDER BY SUM(amount) DESC LIMIT 10
- Q5: Aggregate region + category ก่อน แล้วใช้ RANK() หรือ ROW_NUMBER() PARTITION BY region

- Q6: เน้นให้สังเกตว่า analytical SQL บนข้อมูลหลักล้านแถวยังสามารถตอบโจทย์เชิง interactive ได้บนเครื่องทั่วไป แต่ประสิทธิภาพขึ้นกับอุปกรณ์และ browser

13. Optional: Export Result เป็น CSV

ใน DuckDB Desktop/CLI สามารถ COPY ผล Query ออกเป็นไฟล์ CSV ได้ ส่วน Web Shell อาจมีข้อจำกัดตาม Browser และ session ดังนั้นในห้องเรียนให้เน้นการ Query และ Screenshot เป็นหลัก

```
COPY (  
  SELECT p.category, SUM(s.amount) AS revenue  
  FROM sales s  
  JOIN products p ON s.product_id = p.product_id  
  GROUP BY p.category  
) TO 'category_sales.csv' (HEADER, DELIMITER ',');
```